LangGraph v1.0.x

Section 02: Building Your First Graph

Agentic AI Foundations

March 2026

From Concept to Running Graph

The Five-Step Pattern

1. **Define** state schema (TypedDict)
2. **Create** StateGraph instance
3. **Add** nodes (functions)
4. **Add** edges (control flow)
5. **Compile** and invoke

This same five-step pattern scales from "hello world" to production agents.

Step 1: Define State Schema

TypedDict

```python
from typing_extensions import TypedDict

class State(TypedDict):
    user_input: str
    response: str
```

State Design

Think about what data flows through your graph. Each field is a piece of state.

Step 2: Create StateGraph

Initialize

```python
from langgraph.graph import StateGraph

graph = StateGraph(State)
```

That's it. The StateGraph holds your nodes and edges.

Methods

`graph.add_node()`, `graph.add_edge()`, `graph.compile()`

Step 3 & 4: Add Nodes and Edges

Define a Node

```
def process_input(state: State) -> dict:
    response = state["user_input"].upper()
    return {"response": response}

graph.add_node("process", process_input)
```

Connect with Edges

```
from langgraph.graph import START, END
graph.add_edge(START, "process")
graph.add_edge("process", END)
```

Step 5: Compile and Invoke

Compile

```
compiled_graph = graph.compile()
```

Invoke

```
result = compiled_graph.invoke(
    {"user_input": "hello"}
)
print(result["response"])  # HELLO
```

`invoke()` runs the graph synchronously and returns final state.

StateGraph: Constructor & Methods

Constructor

```
StateGraph(State)  # Takes your state schema
```

Method	Purpose
`add_node(name, fn)`	Add a computation node
`add_edge(src, dst)`	Add a normal edge
`add_conditional_edges()`	Add conditional routing
`compile()`	Finalize and return runnable

Always call `compile()` before `invoke()`.

Adding Nodes: graph.add_node()

Node Registration

`add_node(name, function)` registers a named node.

Multiple Nodes

```
graph.add_node("node_a", func_a)
graph.add_node("node_b", func_b)
graph.add_node("node_c", func_c)
```

Names

Node names are strings. Use them in `add_edge()` calls.

Adding Edges: graph.add_edge()

Normal Edges

```
graph.add_edge(START, "node_a")
graph.add_edge("node_a", "node_b")
graph.add_edge("node_b", END)
```

Order Matters

Data flows from source to destination. Define nodes before adding edges!

Compiling: graph.compile()

What compile() does

- Validates graph structure (reachable, valid edges)
- Returns a *runnable* (has `invoke()`, `stream()`)
- Prepares execution engine

Compile

```
compiled = graph.compile()
# compiled is now a Runnable
```

Compile only once, invoke many times.

Invoking: compiled.invoke()

Execution

`invoke(initial_state)` runs the graph from START to END.

Invoke and Return

```
result = compiled.invoke({
    "user_input": "hello world"
})
print(result)  # Full final state
```

Synchronous

`invoke()` blocks until completion. For async, use `ainvoke()`.

Complete Example: Simple Chatbot

Full Chatbot

```python
from typing_extensions import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages]

def chatbot_node(state: State):
    # Simulate LLM response
    return {"messages": ["Bot: Hello!"]}

graph = StateGraph(State)
graph.add_node("chatbot", chatbot_node)
graph.add_edge(START, "chatbot")
graph.add_edge("chatbot", END)
compiled = graph.compile()

result = compiled.invoke({
    "messages": ["User: Hi"]
})
```

Example: Classification Pipeline

Classifier

```python
class State(TypedDict):
    text: str
    category: str

def classify(state: State) -> dict:
    text = state["text"]
    cat = "spam" if "buy now" in text else "ok"
    return {"category": cat}

graph = StateGraph(State)
graph.add_node("classifier", classify)
graph.add_edge(START, "classifier")
graph.add_edge("classifier", END)
compiled = graph.compile()

result = compiled.invoke(
    {"text": "Check this out"}
)
```

Getting Graph Visualization

Mermaid Diagram

View your graph structure as a flowchart.

Draw Mermaid

```
mermaid_str = compiled.get_graph().draw_mermaid()
print(mermaid_str)
# Paste into mermaid.live to visualize
```

Output

ASCII flowchart showing all nodes and edges. Great for debugging!

Passing Config: RunnableConfig

Config dict

Control graph execution behavior (threading, tags, streaming).

Example

```python
from langgraph.types import RunnableConfig

config = {"tags": ["production"]}
result = compiled.invoke(
    {"text": "test"},
    config=config
)
```

Common config keys: `tags`, `run_name`, `recursion_limit`.

Input and Output Schemas

Automatic

LangGraph infers input/output schema from your State TypedDict.

Inspect Schemas

```
print(compiled.input_schema)
print(compiled.output_schema)
```

Why It Matters

Used for validation, API documentation, and tool-calling in frameworks.

Type Checking and Validation

TypedDict Validation

Use `typing_extensions` for runtime validation.

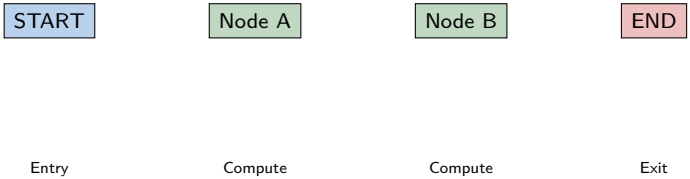
Validation

```
# Invoke with wrong type
result = compiled.invoke({
    "text": 123  # Should be string
})  # May error or coerce
```

Tip

Always define state fields with correct types. Type hints prevent bugs!

Anatomy of a Compiled Graph



Every compiled graph has this structure: START -¿ nodes → END.

Debug Tips: Printing State

Debug Node

```python
def debug_node(state: State) -> dict:
    print(f"State: {state}")
    # Do work
    return {"field": value}

graph.add_node("debug", debug_node)
```

Using stream()

For better debugging, use `stream()` to see intermediate outputs.

Common Graph Patterns

Linear Graph

START -¿ A → B → C → END. Sequential processing.

Branching Graph

START -¿ A → (B or C or D) → END. Uses conditional edges.

Looping Graph

A -¿ (A again or next). Cycles for agent loops.

Section 03 covers branching and looping in detail.

Use mermaid diagram, print state, test with simple inputs first.

Self-Check Questions

b0pt

1. What are the five steps to build a LangGraph?
2. What does StateGraph constructor take?
3. How do you add a node?
4. When must you call compile()?
5. What does invoke() return?
6. How can you visualize your graph?
7. What's the difference between invoke() and stream()?

Quick Answers: (1) Define state, create graph, add nodes, add edges, compile/invoke. (2) State schema. (3) graph.add_node(name, fn). (4) Before invoke(). (5) Final state dict. (6) get_graph().draw_mermaid(). (7) invoke blocks; stream returns events.